

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 Математика и компьютерные науки

Отчёт по лабораторным работам №1–5
по дисциплине «Теория графов»
на тему «Разработка программного комплекса для
генерации графов и реализации базовых алгоритмов
теории графов»

Выполнил студент группы 5130201/40003 _____ Джабраилов А. В.

Проверил _____ Востров А. В.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	4
1 Постановка задачи	5
2 Лабораторная работа №1. Генерация графа и базовый анализ	7
2.1 Постановка задачи	7
2.2 Математическое описание	7
2.3 Особенности реализации	8
3 Лабораторная работа №2. Обход в ширину и кратчайшие пути	14
3.1 Постановка задачи	14
3.2 Математическое описание	14
3.3 Особенности реализации	15
4 Лабораторная работа №3. Сети потоков	18
4.1 Постановка задачи	18
4.2 Математическое описание	18
4.3 Особенности реализации	19
5 Лабораторная работа №4. Остовные деревья, код Прюфера и паросочетания	24
5.1 Постановка задачи	24
5.2 Математическое описание	24
5.3 Особенности реализации	25
6 Лабораторная работа №5. Эйлеровы обходы и цикловая структура графа	29
6.1 Постановка задачи	29
6.2 Математическое описание	29
6.3 Особенности реализации	30
7 Структура программной системы	34
7.1 Общая архитектура	34
7.2 Состав модулей	35
7.3 Пользовательский интерфейс	35

8 Особенности и ограничения текущей реализации	37
Заключение	38

Введение

В рамках цикла лабораторных работ была разработана единая консольная программа на языке C++, объединяющая пять тематических блоков по теории графов. Программа строится вокруг общего графового ядра и позволяет последовательно выполнять генерацию графа, анализ его метрических характеристик, поиск путей, работу с потоками, построение остовов, кодирование деревьев, поиск паросочетаний, а также анализ эйлеровых циклов и базиса циклов.

Практическая ценность такого подхода состоит в том, что результаты предыдущих лабораторных работ не теряются, а переиспользуются в последующих. Например, граф, построенный в первой лабораторной работе, используется при обходе в ширину и поиске кратчайших путей во второй, преобразуется в сеть потоков в третьей, становится основой для построения минимального остова в четвёртой и служит исходными данными для эйлеризации и анализа циклической структуры в пятой.

Особенностью проекта является то, что все лабораторные задания сведены в один исполняемый файл `lab5/src/main.cpp`. По сути, итоговая версия программы представляет собой наращиваемый программный комплекс, в котором каждая следующая лабораторная расширяет уже существующую архитектуру новыми классами, структурами данных и алгоритмами.

1. Постановка задачи

Целью работы являлась реализация набора алгоритмов теории графов и объединение их в единый программный комплекс с текстовым пользовательским интерфейсом. В ходе выполнения лабораторных работ требовалось решить следующие задачи:

Лабораторная работа №1

1. реализовать генерацию графа на основе биномиального распределения;
2. реализовать вычисление эксцентриситетов вершин;
3. реализовать поиск центра графа;
4. реализовать определение диаметральных вершин графа;
5. реализовать метод Шимбелла;
6. реализовать поиск всех простых путей между двумя вершинами.

Лабораторная работа №2

7. реализовать обход графа в ширину;
8. реализовать классический алгоритм Дейкстры;
9. реализовать сравнение алгоритмов по числу итераций.

Лабораторная работа №3

10. реализовать построение сети потоков по ориентированному графу;
11. реализовать поиск максимального потока;
12. реализовать поиск потока минимальной стоимости.

Лабораторная работа №4

13. реализовать вычисление числа остовных деревьев по теореме Кирхгофа;

14. реализовать построение минимального остова алгоритмом Краскала;
15. реализовать кодирование дерева кодом Прюфера;
16. реализовать декодирование дерева по коду Прюфера;
17. реализовать поиск паросочетания.

Лабораторная работа №5

18. реализовать проверку графа на эйлеровость;
19. реализовать эйлеризацию графа при необходимости;
20. реализовать построение эйлера обхода;
21. реализовать построение фундаментальной системы циклов;
22. реализовать операцию симметрической разности циклов.

Помимо собственно алгоритмов, требовалось реализовать пользовательское меню, проверки входных данных и повторное использование уже построенных структур в рамках одного сеанса работы программы.

2. Лабораторная работа №1. Генерация графа и базовый анализ

2.1. Постановка задачи

Первая лабораторная работа посвящена построению исходного графа и базовым алгоритмам его анализа. Необходимо сгенерировать граф с числом вершин, заданным пользователем, распределить степени и веса по биномиальному закону, затем вычислить эксцентриситеты вершин, центр и диаметр графа, реализовать метод Шимбелла для путей фиксированной длины и определить количество маршрутов между двумя вершинами.

2.2. Математическое описание

В лекциях граф определяется как совокупность двух множеств $G(V, E)$, где V — непустое множество вершин, а E — множество двухэлементных подмножеств множества V . Если элементами E являются упорядоченные пары, то граф является ориентированным, и тогда элементы множества E называются дугами. Вершины, инцидентные одному ребру, называются смежными, а множество вершин, смежных с вершиной v , образует её множество смежности.

Степенью вершины $d(v)$ называется число инцидентных ей рёбер. Для неориентированного графа справедлива лемма о рукопожатиях: сумма степеней всех вершин равна удвоенному числу рёбер, поэтому число вершин нечётной степени всегда чётно. Именно это свойство учитывается при генерации неориентированного графа.

В работе используется помеченный граф, то есть граф, у которого рёбрам сопоставлены веса. Случайные значения степеней и весов формируются на основе биномиального распределения с параметрами n и p .

Маршрутом в графе называется чередующаяся последовательность вершин и рёбер, в которой любые два соседних элемента инцидентны. Если все рёбра маршрута различны, то такой маршрут называется цепью, а если все его вершины различны, то он называется простой цепью. Замкнутая цепь называется циклом, а замкнутая простая цепь — простым циклом. Граф без циклов называется ациклическим.

Две вершины графа называются связанными, если существует соединяющая их простая цепь. Граф, в котором все вершины связаны между собой, называется связным. Число компонент связности графа G обозначается $k(G)$. Расстоянием $d(u, v)$ между вершинами u и v называется длина кратчайшей цепи между ними, измеряемая числом рёбер.

Эксцентриситетом $e(v)$ вершины v в связном графе называется максимальное расстояние от этой вершины до других вершин графа:

$$e(v) = \max_{u \in V} d(v, u).$$

Радиус графа определяется формулой

$$R(G) = \min_{v \in V} e(v),$$

Вершина называется центральной, если её эксцентриситет совпадает с радиусом графа, а множество всех таких вершин образует центр графа. Наиболее эксцентричные вершины являются концами диаметра.

Метод Шимбелла в лекциях используется для выявления маршрутов с заданным количеством рёбер. В лекционном изложении для определения количества маршрутов, состоящих из k рёбер, требуется возвести в k -ую степень матрицу смежности вершин; соответствующий элемент полученной матрицы даёт количество маршрутов длины k . В программе эта идея адаптирована к взвешенному графу: вместо простого подсчёта числа маршрутов дополнительно вычисляются минимальные и максимальные суммарные веса маршрутов фиксированной длины.

Поиск всех путей между двумя вершинами в терминах лекций соответствует поиску всех простых цепей между этими вершинами, то есть всех последовательностей смежных вершин с фиксированными началом и концом, в которых вершины не повторяются.

2.3. Особенности реализации

Ключевой модуль генерации реализован в классе `AcyclicGraphBuilder`. В нём метод `sampleBinomial` вычисляет случайную величину с биномиальным распределением без обращения к готовому

распределению стандартной библиотеки. Метод `generateAcyclicGraph` сначала генерирует целевые степени всех вершин, затем строит рёбра и после этого проверяет связность графа. Если граф оказывается несвязным, генерация повторяется. В коде предусмотрено до 50 попыток регенерации.

Для основных функций этого модуля вход и выход можно описать следующим образом.

Метод `sampleBinomial(n, p)`

Вход: параметры биномиального распределения n и p , а также внутреннее состояние генератора случайных чисел класса.

Выход: одно целое случайное значение, распределённое по биномиальному закону.

Описание: генерирует случайное число по биномиальному распределению. Код представлен в листинге 1.

```
int sampleBinomial(int n, double p);
```

Листинг 1: Фрагмент функции `sampleBinomial(n, p)`

Метод `generateAcyclicGraph(vertices, directed, weightSign)`

Вход: количество вершин, признак ориентированности графа, режим генерации весов, а также внутренние параметры биномиального распределения, закреплённые в классе.

Выход: указатель на сгенерированный граф, содержащий вершины, рёбра и веса.

Описание: строит граф с заданным числом вершин и режимом генерации весов.

Код представлен в листинге 2.

```
std::unique_ptr<AdjacencyGraph> generateAcyclicGraph(  
    int vertexCount,  
    bool directed,  
    WeightSign weightSign  
);
```

Листинг 2: Фрагмент функции `generateAcyclicGraph(vertices, directed, weightSign)`

Метод `computeDegreeStatistics(graph)`

Вход: уже построенный граф.

Выход: структура со средним значением степени, дисперсией, стандартным

отклонением, минимальной и максимальной степенью.

Описание: вычисляет основные статистики по степеням вершин графа.

Код представлен в листинге 3.

```
static DegreeStatistics computeDegreeStatistics(  
    const AdjacencyGraph& graph  
);
```

Листинг 3: Фрагмент функции `computeDegreeStatistics(graph)`

В ориентированном режиме рёбра добавляются только между вершинами $u < v$, что исключает появление ориентированных циклов и даёт ориентированный ациклический граф. В неориентированном режиме используется та же схема выбора концов ребра, однако сама по себе она не является строгим доказательством отсутствия циклов; фактически она задаёт порядок генерации и исключает петли и дубликаты.

Вычисление эксцентриситетов реализовано в классе `GraphAnalyzer`. Для каждой стартовой вершины выполняется BFS, после чего формируется таблица расстояний по числу рёбер. Далее по этим расстояниям вычисляются радиус, диаметр, центральные вершины и все диаметральные пути.

Для этого класса важны две функции.

Метод `computeEdgeCountPaths()`

Вход: граф, сохранённый во внутреннем состоянии объекта `GraphAnalyzer`.

Выход: таблица расстояний между всеми упорядоченными парами вершин, где расстояние измеряется числом рёбер, а для недостижимых пар фиксируется специальное значение.

Описание: вычисляет расстояния между всеми парами вершин по числу рёбер.

Код представлен в листинге 4.

```
std::map<std::pair<int, int>, int> computeEdgeCountPaths() const;
```

Листинг 4: Фрагмент функции `computeEdgeCountPaths()`

Метод `analyze()`

Вход: граф, переданный объекту при создании класса.

Выход: структура `EccentricityData`, содержащая эксцентриситеты, радиус, диаметр, центр графа, диаметральные вершины и набор диаметральных путей.

Описание: находит основные метрические характеристики графа.

Код представлен в листинге 5.

```
EccentricityData analyze();
```

Листинг 5: Фрагмент функции `analyze()`

Метод Шимбелла реализован в классе `KPathCalculator`. Сначала строится базовая матрица весов для путей длины 1, затем она последовательно перемножается сама на себя в полукольцах $(\min, +)$ и $(\max, +)$. При отсутствии пути используется значение `std::nullopt`, что позволяет явно отделять «пути нет» от нулевого веса.

С точки зрения входа и выхода основные функции здесь таковы.

Метод `buildWeightMatrix()`

Вход: граф, сохранённый внутри объекта `KPathCalculator`.

Выход: базовая матрица весов для путей длины 1.

Описание: строит матрицу весов для путей, состоящих из одного ребра.

Код представлен в листинге 6.

```
WeightTable buildWeightMatrix() const;
```

Листинг 6: Фрагмент функции `buildWeightMatrix()`

Метод `multiplyMinPlus(left, right)`

Вход: две матрицы весов путей.

Выход: матрица минимальных суммарных весов при композиции путей.

Описание: перемножает две матрицы по правилу $(\min, +)$.

Код представлен в листинге 7.

```
WeightTable multiplyMinPlus(  
    const WeightTable& left,  
    const WeightTable& right  
) const;
```

Листинг 7: Фрагмент функции `multiplyMinPlus(left, right)`

Метод `multiplyMaxPlus(left, right)`

Вход: две матрицы весов путей.

Выход: матрица максимальных суммарных весов при композиции путей.

Описание: перемножает две матрицы по правилу $(\max, +)$.

Код представлен в листинге 8.

```
WeightTable multiplyMaxPlus(  
    const WeightTable& left,  
    const WeightTable& right  
) const;
```

Листинг 8: Фрагмент функции multiplyMaxPlus(left, right)

Метод compute(edgeCount)

Вход: требуемое число рёбер k и граф, связанный с объектом класса.

Выход: структура ShimbellOutput, содержащая матрицу минимальных весов, матрицу максимальных весов и само значение k .

Описание: вычисляет минимальные и максимальные веса путей длины k .

Код представлен в листинге 9.

```
ShimbellOutput compute(int edgeCount);
```

Листинг 9: Фрагмент функции compute(edgeCount)

Поиск всех простых путей вынесен в класс SimplePathFinder. Он использует рекурсивный обход в глубину с массивом посещённых вершин и текущим накапливаемым путём. При достижении целевой вершины найденный путь сохраняется в коллекцию.

Для этого алгоритма удобно явно зафиксировать следующее.

Метод findAllPaths(from, to)

Вход: номера начальной и конечной вершин, а также граф, сохранённый в объекте SimplePathFinder.

Выход: коллекция всех найденных простых путей между указанными вершинами.

Описание: находит все простые пути между двумя вершинами.

Код представлен в листинге 10.

```
PathCollection findAllPaths(int from, int to);
```

Листинг 10: Фрагмент функции findAllPaths(from, to)

Метод countPaths(from, to)

Вход: те же две вершины и граф, связанный с объектом класса.

Выход: количество простых путей между ними.

Описание: подсчитывает число простых путей между двумя вершинами.

Код представлен в листинге 11.

```
int countPaths(int from, int to);
```

Листинг 11: Фрагмент функции countPaths(from, to)

Метод dfsExplore(current, target, visited, path, allPaths)

Вход: текущая вершина обхода, целевая вершина, массив посещённых вершин, текущий путь и контейнер для всех найденных путей.

Выход: модифицированный контейнер allPaths, в который добавляются найденные пути; отдельного возвращаемого значения функция не имеет.

Описание: рекурсивно расширяет текущий путь и сохраняет найденные маршруты.

Код представлен в листинге 12.

```
void dfsExplore(  
    int current,  
    int target,  
    std::unordered_set<int>& visited,  
    std::vector<int>& path,  
    PathCollection& allPaths  
);
```

Листинг 12: Фрагмент функции dfsExplore(current, target, visited, path, allPaths)

В пользовательском интерфейсе первой лабораторной работе соответствуют пункты 1–8. Пользователь может сгенерировать граф, вывести матрицу смежности, матрицу весов для случая $k = 1$, статистику по степеням и результаты всех перечисленных алгоритмов.

3. Лабораторная работа №2. Обход в ширину и кратчайшие пути

3.1. Постановка задачи

Во второй лабораторной работе на уже построенном графе необходимо выполнить обход в ширину, найти кратчайший путь между двумя вершинами классическим алгоритмом Дейкстры и сравнить оба алгоритма по числу итераций.

3.2. Математическое описание

В лекциях поиск в ширину рассматривается как алгоритм Мура. Он строит слоистую структуру графа относительно стартовой вершины s : на нулевом уровне находится сама вершина s , на первом — все вершины, смежные с s , на втором — вершины, достижимые цепью длины 2, и так далее. Поэтому алгоритм Мура находит кратчайшие цепи в графе, если длина цепи измеряется числом рёбер.

Алгоритм Дейкстры, согласно лекциям, находит оптимальные маршруты и их длину между одной конкретной вершиной-источником и всеми остальными вершинами графа. Он корректен для графов и орграфов без рёбер отрицательного веса. Недостаток алгоритма состоит в том, что при наличии отрицательных весов он может работать некорректно.

В классической табличной форме алгоритма Дейкстры используются следующие обозначения:

- $T[v]$ — текущая метка вершины v , то есть текущая оценка длины маршрута от источника до v ;
- $X[v]$ — признак того, что метка вершины стала постоянной;
- $H[v]$ — предшественник вершины v в дереве кратчайших путей.

На каждом шаге выбирается вершина с минимальной временной меткой, после чего выполняется корректировка меток для смежных с ней вершин. После

завершения работы алгоритма по массиву предшественников восстанавливается дерево кратчайших путей и, при необходимости, конкретный маршрут до выбранной вершины.

Таким образом, вторая лабораторная работа сопоставляет два разных критерия длины цепи:

- в алгоритме Мура длина измеряется количеством рёбер;
- в алгоритме Дейкстры длина определяется суммой весов рёбер маршрута.

Если требуется обрабатывать графы с произвольными весами, включая отрицательные, то в лекциях для этого предлагается алгоритм Беллмана–Форда, однако в данной лабораторной работе реализуется именно классический алгоритм Дейкстры.

3.3. Особенности реализации

Обход в ширину реализован в классе `BreadthFirstSearch`. Метод `traverseWithLevels` хранит:

- порядок обхода вершин;
- отображение уровня в список вершин этого уровня;
- число выполненных итераций;
- максимальную глубину обхода.

Таким образом, пользователь получает не только факт достижимости, но и слоистое представление графа.

Формально вход и выход функции можно описать так.

Метод `traverseWithLevels(startVertex)`

Вход: стартовая вершина и граф, заранее сохранённый во внутреннем состоянии класса `BreadthFirstSearch`.

Выход: структура `BFSResult`, содержащая порядок обхода, уровни, число итераций и максимальную глубину.

Описание: выполняет обход в ширину и определяет уровни вершин.

Код представлен в листинге 13.

```
BFSResult traverseWithLevels(int startVertex);
```

Листинг 13: Фрагмент функции `traverseWithLevels(startVertex)`

Алгоритм Дейкстры реализован в классе `DijkstraAlgorithm`. В коде используется классическая схема без очереди с приоритетом: на каждой итерации линейным просмотром выбирается следующая вершина с минимальной меткой. Это делает реализацию прозрачной и хорошо соответствующей учебному варианту алгоритма. Дополнительно сохраняются предшественники и формируется вектор расстояний от исходной вершины до всех достижимых вершин.

При описании функций этого класса удобно использовать следующий формат.

Метод `findShortestPaths(startVertex, targetVertex)`

Вход: стартовая вершина, необязательная целевая вершина и граф, связанный с объектом `DijkstraAlgorithm`.

Выход: структура `DijkstraResult`, содержащая расстояния до достижимых вершин, предшественников, восстановленный кратчайший путь до цели, итоговое расстояние и число итераций.

Описание: находит кратчайшие пути от стартовой вершины алгоритмом Дейкстры.

Код представлен в листинге 14.

```
DijkstraResult findShortestPaths(  
    int startVertex,  
    std::optional<int> targetVertex = std::nullopt  
);
```

Листинг 14: Фрагмент функции `findShortestPaths(startVertex, targetVertex)`

Метод `reconstructPath(startVertex, targetVertex, H, vertexIds)`

Вход: начальная вершина, конечная вершина, массив предшественников и список идентификаторов вершин.

Выход: последовательность вершин, задающая найденный кратчайший путь; если путь не существует, возвращается пустой вектор.

Описание: восстанавливает путь по массиву предшественников.

Код представлен в листинге 15.

```
std::vector<int> reconstructPath(  
    int startVertex,  
    int targetVertex,  
    const std::vector<int>& H,  
    const std::vector<int>& vertexIds  
);
```

Листинг 15: Фрагмент функции `reconstructPath(startVertex, targetVertex, H, vertexIds)`

Сравнение алгоритмов вынесено в класс `AlgorithmComparator`. Он запускает BFS и алгоритм Дейкстры из одной и той же вершины и сравнивает их по количеству учтённых операций. В программе выводится отношение числа итераций алгоритма Дейкстры к числу итераций BFS.

Соответствующая функция описывается так.

Метод `compare(startVertex)`

Вход: стартовая вершина и граф, сохранённый в объекте `AlgorithmComparator`.

Выход: структура `AlgorithmComparison`, содержащая число итераций BFS, число итераций алгоритма Дейкстры и коэффициент сравнения.

Описание: сравнивает BFS и алгоритм Дейкстры по числу итераций.

Код представлен в листинге 16.

```
AlgorithmComparison compare(int startVertex);
```

Листинг 16: Фрагмент функции `compare(startVertex)`

Практически важно отметить, что корректность алгоритма Дейкстры обеспечивается только при неотрицательных весах рёбер. Поскольку генератор программы умеет создавать и отрицательные веса, при запуске этой части работы желательно использовать положительный режим генерации весов. Сам код не запрещает запуск алгоритма на смешанных весах, поэтому это ограничение следует учитывать при интерпретации результатов.

Пользовательские пункты второй лабораторной работы — 9, 10 и 11. Они позволяют вывести уровни BFS, кратчайший путь между двумя вершинами и сравнение скорости работы алгоритмов.

4. Лабораторная работа №3. Сети потоков

4.1. Постановка задачи

Третья лабораторная работа основана на ориентированном графе. Требуется случайным образом сформировать матрицы пропускных способностей и стоимостей, построить по ним сеть потоков, найти максимальный поток между выбранными вершинами и затем вычислить поток минимальной стоимости величины, равной двум третям от найденного максимального потока с округлением вниз.

4.2. Математическое описание

С точки зрения лекций сеть потоков задаётся ориентированным графом, в котором по каждой дуге протекает некоторый поток. Каждая дуга характеризуется тремя параметрами:

- нижней границей потока;
- пропускной способностью, то есть верхней границей потока;
- стоимостью передачи единицы потока по данной дуге.

В реализованной модели нижняя граница явно не задаётся, поэтому по умолчанию считается равной нулю.

Если через $f(u, v)$ обозначить поток по дуге (u, v) , через $c(u, v)$ — её пропускную способность, а через $p(u, v)$ — стоимость единицы потока, то поток по каждой дуге должен удовлетворять ограничениям, задаваемым нижней и верхней границами, а в промежуточных вершинах должен выполняться закон сохранения потока.

Максимальный поток ищется на основе теоремы и алгоритма Форда–Фалкерсона. Рассматривается остаточная сеть: если по некоторой дуге ещё можно увеличить поток, то в остаточной сети присутствует прямая дуга; если часть потока можно вернуть назад, то появляется обратная дуга. Наличие увеличивающего пути из истока в сток означает, что значение потока можно увеличить на величину минимальной остаточной пропускной способности вдоль этого пути.

Поток минимальной стоимости в лекциях рассматривается как задача потокового программирования. Требуется выбрать значения потоков по дугам так, чтобы удовлетворялись ограничения по пропускным способностям и сохранению потока, а суммарная стоимость передачи потока была минимальной. В программе для этого используется последовательный поиск кратчайших по стоимости увеличивающих путей в остаточной сети. Сначала определяется величина максимального потока, после чего строится поток минимальной стоимости величины, равной двум третям от найденного максимального потока с округлением вниз.

4.3. Особенности реализации

Построение сети потоков вынесено в класс `FlowNetworkBuilder`. Он не меняет структуру исходного ориентированного графа, а только копирует его топологию. Пропускные способности и стоимости затем генерируются заново с использованием того же генератора биномиального распределения, что и в первой лабораторной работе.

Основная функция этого модуля имеет следующий интерфейс в логическом смысле.

Метод `buildFromGraph(graph)`

Вход: ориентированный граф, задающий топологию сети.

Выход: новая сеть потоков, в которой вершины и дуги унаследованы от исходного графа, а пропускные способности и стоимости сгенерированы автоматически.

Описание: строит сеть потоков по ориентированному графу.

Код представлен в листинге 17.

```
std::unique_ptr<FlowNetwork> buildFromGraph(  
    const AdjacencyGraph& graph  
);
```

Листинг 17: Фрагмент функции `buildFromGraph(graph)`

Класс `FlowNetwork` хранит реальные дуги и умеет на лету строить остаточные дуги. Для этого вводится структура `ResidualArc`, которая содержит направление, остаточную пропускную способность, стоимость и ссылку на исходную дугу. Благодаря этому один и тот же сетевой объект

используется и для максимального потока, и для потока минимальной стоимости.

Для используемых в отчёте функций этого класса.

Метод `residualNeighbors(vertex)`

Вход: вершина сети потоков и текущее состояние всех потоков в объекте `FlowNetwork`.

Выход: список остаточных дуг, по которым поток можно увеличить или уменьшить.

Описание: возвращает доступные переходы из вершины в остаточной сети. Код представлен в листинге 18.

```
std::vector<ResidualArc> residualNeighbors(int vertex) const;
```

Листинг 18: Фрагмент функции `residualNeighbors(vertex)`

Метод `getCapacityMatrix()`, `getCostMatrix()`, `getFlowMatrix()`

Вход: текущее состояние сети потоков.

Выход: соответствующие матричные представления пропускных способностей, стоимостей и текущего потока.

Описание: возвращает матрицы пропускных способностей, стоимостей и текущего потока.

Код представлен в листинге 19.

```
FlowMatrix getCapacityMatrix() const;  
FlowMatrix getCostMatrix() const;  
FlowMatrix getFlowMatrix() const;
```

Листинг 19: Фрагмент функций получения матриц сети потока

Метод `augment(edgeIndex, forward, delta)`

Вход: индекс исходной дуги, признак прямого или обратного прохода и величина изменения потока.

Выход: обновлённое внутреннее состояние сети; отдельного возвращаемого значения функция не имеет.

Описание: изменяет поток по выбранной дуге на заданную величину.

Код представлен в листинге 20.

```
void augment(int edgeIndex, bool forward, int delta);
```

Листинг 20: Фрагмент функции `augment(edgeIndex, forward, delta)`

Максимальный поток реализован в классе `MaxFlowSolver`. По сути, это вариант алгоритма Форда–Фалкерсона с поиском увеличивающего пути в ширину, то есть схема, близкая к алгоритму Эдмондса–Карпа. После каждого успешного шага сохраняются:

- номер шага;
- найденный путь по вершинам;
- величина добавленного потока;
- суммарный поток после увеличения.

Это делает вычисление удобным для трассировки и проверки.

В терминах функций описание выглядит так.

Метод `compute(source, sink)`

Вход: вершины истока и стока, а также сеть потоков, связанная с объектом `MaxFlowSolver`.

Выход: структура `MaxFlowResult`, содержащая значение максимального потока и последовательность увеличивающих шагов.

Описание: находит максимальный поток между истоком и стоком.

Код представлен в листинге 21.

```
MaxFlowResult compute(int source, int sink);
```

Листинг 21: Фрагмент функции `compute(source, sink)` для максимального потока

Метод `bfs(source, sink, parent)`

Вход: исток, сток, остаточная сеть, задаваемая текущим состоянием объекта `FlowNetwork`, и контейнер для предков.

Выход: логическое значение, показывающее, найден ли увеличивающий путь; дополнительно по ссылке заполняется структура предков, описывающая этот путь.

Описание: ищет увеличивающий путь в остаточной сети.

Код представлен в листинге 22.

```
bool bfs(  
    int source,  
    int sink,
```

```
std::vector<std::optional<ParentArc>>& parent  
);
```

Листинг 22: Фрагмент функции bfs(source, sink, parent)

Поток минимальной стоимости реализован в классе `MinCostFlowSolver`. В нём на остаточной сети выполняется поиск кратчайшего по стоимости пути, после чего по нему проводится очередное увеличение потока. В программе целевая величина потока вычисляется автоматически как две трети от найденного ранее максимального потока с округлением вниз.

Здесь ключевые функции имеют такой вход и выход.

Метод `compute(source, sink, targetFlow)`

Вход: исток, сток, требуемая величина потока и сеть потоков, закреплённая за объектом `MinCostFlowSolver`.

Выход: структура `MinCostFlowResult`, содержащая целевой и достигнутый поток, полную стоимость и пошаговую историю увеличений.

Описание: находит поток минимальной стоимости заданной величины.

Код представлен в листинге 23.

```
MinCostFlowResult compute(int source, int sink, int targetFlow);
```

Листинг 23: Фрагмент функции compute(source, sink, targetFlow)

Метод `dijkstra(source, sink, distances, parent)`

Вход: исток, сток, текущее состояние остаточной сети и контейнеры для расстояний и предков.

Выход: логическое значение, показывающее существование допустимого пути минимальной стоимости; дополнительно заполняются отображения расстояний и дуг-предков.

Описание: ищет кратчайший по стоимости путь в остаточной сети.

Код представлен в листинге 24.

```
bool dijkstra(  
    int source,  
    int sink,  
    std::vector<int>& distances,  
    std::vector<std::optional<ParentArc>>& parent  
);
```

Листинг 24: Фрагмент функции `dijkstra(source, sink, distances, parent)`

В пользовательском меню лабораторной работе №3 соответствуют пункты 12–17. Пользователь сначала строит сеть потоков, затем может вывести матрицы пропускных способностей и стоимостей, найти максимальный поток, а после этого — поток минимальной стоимости для той же пары «исток–сток».

5. Лабораторная работа №4. Остовные деревья, код Прюфера и паросочетания

5.1. Постановка задачи

Четвёртая лабораторная работа выполняется на неориентированном графе. Требуется вычислить число остовных деревьев по матричной теореме Кирхгофа, построить минимальный остов алгоритмом Краскала, закодировать и декодировать полученное дерево кодом Прюфера, а также определить независимое множество рёбер.

5.2. Математическое описание

Пусть $G(V, E)$ — неориентированный граф. Остовным подграфом графа G называется подграф, содержащий все вершины графа. Остовный подграф, являющийся деревом, называется остовом, или каркасом. Несвязный граф остова не имеет, а связный граф может иметь несколько различных остовов.

Если A — матрица смежности графа, а D — диагональная матрица степеней его вершин, то матрица Кирхгофа определяется формулой

$$B(G) = D - A.$$

По теореме Кирхгофа число остовных деревьев в связном графе порядка $n \geq 2$ равно алгебраическому дополнению любого элемента матрицы $B(G)$. Иными словами, достаточно вычислить любой её кофактор, то есть определитель минора порядка $n - 1$.

Если рёбрам графа приписаны веса, то возникает задача построения кратчайшего остова, то есть остова минимального суммарного веса. В лабораторной работе для этого используется алгоритм Краскала: рёбра перебираются в порядке возрастания их веса, и каждое безопасное ребро добавляется в остов. Безопасным является ребро, добавление которого не образует цикла с уже выбранными рёбрами. В лекциях для этого алгоритма приводится оценка трудоёмкости $O(E \log E)$.

Код Прюфера в лекциях рассматривается как экономное по памяти

представление свободного дерева. Для дерева $T(V, E)$ с вершинами, занумерованными числами от 1 до p , строится последовательность, получаемая последовательным удалением листьев и записью их соседей. В теоретической постановке информация о последнем ребре восстанавливается однозначно. В реализованной версии, помимо самой последовательности, дополнительно сохраняются веса соответствующих рёбер, чтобы можно было декодировать не только структуру дерева, но и его взвешенный вариант.

Паросочетанием называется множество рёбер, не имеющих общих вершин. Если к паросочетанию нельзя добавить ни одного нового ребра, то оно является максимальным по включению. Если же среди всех паросочетаний выбрано паросочетание наибольшей мощности, то оно называется паросочетанием максимальной мощности. В данной работе рассматриваются оба варианта: жадное максимальное по включению паросочетание и точный поиск паросочетания максимальной мощности алгоритмом Эдмондса.

5.3. Особенности реализации

Класс `KirchhoffCounter` строит матрицу Кирхгофа $B(G) = D - A$, а затем вычисляет детерминант минора, полученного удалением первой строки и первого столбца. Для вычисления детерминанта используется метод Гаусса с выбором главного элемента. После вычисления результат округляется, поскольку теоретически он обязан быть целым неотрицательным числом.

С точки зрения входа и выхода здесь используются следующие функции.

Метод `buildKirchhoffMatrix()`

Вход: граф, сохранённый в объекте `KirchhoffCounter`.

Выход: матрица Кирхгофа $B(G) = D - A$.

Описание: строит матрицу Кирхгофа для заданного графа.

Код представлен в листинге 25.

```
std::vector<std::vector<int>>> buildKirchhoffMatrix() const;
```

Листинг 25: Фрагмент функции `buildKirchhoffMatrix()`

Метод `determinantOfFirstMinor(matrix)`

Вход: матрица Кирхгофа.

Выход: целое число, равное детерминанту выбранного минора.

Описание: вычисляет определитель минора матрицы Кирхгофа.

Код представлен в листинге 26.

```
long long determinantOfFirstMinor(  
    const std::vector<std::vector<int>>& matrix  
) const;
```

Листинг 26: Фрагмент функции `determinantOfFirstMinor(matrix)`

Метод `compute()`

Вход: граф, связанный с объектом класса.

Выход: структура `KirchhoffResult`, содержащая матрицу Кирхгофа и число остовных деревьев.

Описание: вычисляет число остовных деревьев графа.

Код представлен в листинге 27.

```
KirchhoffResult compute();
```

Листинг 27: Фрагмент функции `compute()` класса `KirchhoffCounter`

Построение минимального остова реализовано в классе `KruskalMST`. Все рёбра сортируются по весу, после чего проходит стандартный алгоритм Краскала на основе структуры `DisjointSetUnion`. Если граф оказывается несвязным, программа возвращает остовный лес и явно сообщает об этом пользователю.

В этом модуле описание выглядит так.

Метод `buildMST()`

Вход: граф, сохранённый в объекте `KruskalMST`.

Выход: структура `KruskalResult`, содержащая построенный остов, список выбранных рёбер, суммарный вес и признак связности исходного графа.

Описание: строит минимальный остов алгоритмом Краскала.

Код представлен в листинге 28.

```
KruskalResult buildMST() const;
```

Листинг 28: Фрагмент функции `buildMST()`

Класс `PruferEncoder` реализует согласованный внутри проекта вариант кода Прюфера. При кодировании на каждом шаге удаляется лист с минимальным номером, в код записывается его единственный сосед, а в параллельный массив — вес соответствующего ребра. При декодировании по

оставшейся части кода выбирается минимальная неиспользованная вершина, не встречающаяся в хвосте последовательности, после чего восстанавливается очередное ребро.

Функции этого класса описываются так.

Метод encode(tree)

Вход: дерево, представленное объектом AdjacencyGraph.

Выход: структура PruferCode, содержащая последовательность кода и параллельный массив весов.

Описание: кодирует дерево в код Прюфера.

Код представлен в листинге 29.

```
static PruferCode encode(const AdjacencyGraph& tree);
```

Листинг 29: Фрагмент функции encode(tree)

Метод decode(pruferCode, vertexIds)

Вход: код Прюфера с весами и полный список идентификаторов вершин исходного дерева.

Выход: заново построенное дерево как объект AdjacencyGraph.

Описание: восстанавливает дерево по коду Прюфера.

Код представлен в листинге 30.

```
static std::unique_ptr<AdjacencyGraph> decode(  
    const PruferCode& pruferCode,  
    const std::vector<int>& vertexIds  
);
```

Листинг 30: Фрагмент функции decode(pruferCode, vertexIds)

Для поиска независимого множества рёбер реализованы два подхода. Класс GreedyMaximalMatching последовательно просматривает рёбра в лексикографическом порядке и добавляет ребро, если обе его вершины пока не покрыты. Класс EdmondsMatching реализует blossom-алгоритм Эдмондса и позволяет находить уже не просто максимальное по включению, а максимальное по мощности паросочетание в общем неориентированном графе.

Логический вход и выход этих функций таков.

Метод GreedyMaximalMatching::compute()

Вход: граф, связанный с объектом класса.

Выход: структура `MatchingResult`, содержащая выбранные рёбра, непокрытые вершины, размер паросочетания и признак его совершенства.

Описание: строит жадное максимальное по включению паросочетание.

Код представлен в листинге 31.

```
MatchingResult compute() const;
```

Листинг 31: Фрагмент функции `GreedyMaximalMatching::compute()`

Метод `EdmondsMatching::compute()`

Вход: граф, сохранённый во внутреннем состоянии объекта `EdmondsMatching`.

Выход: структура `MatchingResult`, но уже для паросочетания максимальной мощности.

Описание: находит паросочетание максимальной мощности.

Код представлен в листинге 32.

```
MatchingResult compute() const;
```

Листинг 32: Фрагмент функции `EdmondsMatching::compute()`

Пункты 18–23 пользовательского меню позволяют вывести матрицу Кирхгофа, построить и сохранить минимальный остов, показать его рёбра, выполнить кодирование и декодирование деревьев, а также сравнить жадное и точное паросочетания на исходном графе или на построенном остове.

6. Лабораторная работа №5. Эйлеровы обходы и цикловая структура графа

6.1. Постановка задачи

Пятая лабораторная работа посвящена эйлеровым графам и пространству циклов. Требуется определить, является ли неориентированный граф эйлеровым, при необходимости модифицировать его, построить эйлеров обход, а затем исследовать циклическую структуру графа относительно построенного остова.

6.2. Математическое описание

Если граф имеет цикл, содержащий все рёбра графа, то такой цикл называется эйлеровым циклом, а сам граф — эйлеровым. Если граф имеет цепь, содержащую все рёбра графа, то такая цепь называется эйлеровой цепью, а граф — полуэйлеровым. В лекциях отдельно подчёркивается, что эйлеровым может быть только связный граф.

Для связного неориентированного графа условие эйлеровости формулируется стандартно: граф является эйлеровым тогда и только тогда, когда степени всех его вершин чётны. Если же ровно две вершины имеют нечётную степень, то граф является полуэйлеровым и допускает эйлерову цепь, но не цикл.

Пусть $T(V, E_T)$ — некоторый остов связного графа $G(V, E)$. Остовный подграф, образованный рёбрами $E \setminus E_T$, называется кодеревом, а его рёбра — хордами остова. Каждая хорда остова порождает ровно один простой цикл Z_e . Система всех таких циклов называется фундаментальной системой циклов.

Число циклов в фундаментальной системе называется циклическим рангом, или цикломатическим числом, графа G и обозначается $m(G)$. Для связного графа оно равно

$$m(G) = |E| - |V| + 1.$$

Фундаментальная система циклов является базисом подпространства циклов.

Двойственным понятием является разрез. Разрезом связного графа называется множество рёбер, удаление которых делает граф несвязным. Правильный разрез, определяемый непустым подмножеством вершин $U \subset V$, обозначается $S(U)$. Простым разрезом называется минимальный разрез. В кодовой базе присутствует и модуль фундаментальной системы разрезов, однако в пользовательской части пятой лабораторной работы основной акцент сделан именно на фундаментальной системе циклов.

Сложению в пространстве циклов соответствует операция симметрической разности множеств рёбер. Согласно лекциям совокупность всех симметрических разностей фундаментальных циклов содержит не только все циклы графа, но и некоторые объединения циклов. Поэтому результат симметрической разности двух циклов может быть как одним циклом, так и объединением нескольких циклов, то есть циклическим вектором.

6.3. Особенности реализации

Проверка эйлеровости и построение обхода реализованы в классе `EulerianCycleBuilder`. Сначала входной граф копируется, чтобы не менять исходные данные. Затем выполняются два этапа модификации:

1. соединение компонент связности, содержащих рёбра;
2. попарное исправление нечётных степеней путём добавления новых рёбер.

Каждое добавленное ребро заносится в журнал модификаций. Если исправить нечётность без кратных рёбер невозможно, программа сообщает об этом и предлагает пользователю разрешить переход к мультиграфу. После завершения эйлеризации запускается алгоритм Хиерхольцера, и пользователь получает последовательность вершин эйлерова обхода.

Метод `compute(mode)`

Вход: исходный неориентированный граф, закреплённый за объектом `EulerianCycleBuilder`, и режим эйлеризации, разрешающий или запрещающий мультиграф.

Выход: структура `EulerianCycleResult`, содержащая сведения о необходимости модификаций, журнал добавленных рёбер, модифицированный

граф и итоговый эйлеров обход.

Описание: проверяет граф, при необходимости эйлеризует его и строит эйлеров обход.

Код представлен в листинге 33.

```
EulerianCycleResult compute(  
    EulerizationMode mode  
) const;
```

Листинг 33: Фрагмент функции `compute(mode)`

Метод `hierholzer(g, start)`

Вход: граф, уже удовлетворяющий условиям существования эйлерова обхода, и стартовая вершина.

Выход: последовательность вершин эйлерова цикла или пути.

Описание: строит эйлеров цикл или путь алгоритмом Иерихольцера.

Код представлен в листинге 34.

```
std::vector<int> hierholzer(  
    const AdjacencyGraph& g,  
    int start  
);
```

Листинг 34: Фрагмент функции `hierholzer(g, start)`

Для исследования цикловой структуры используется класс `FundamentalCycleSystem`. На вход он получает исходный граф и ранее построенный минимальный остов. Для каждой хорды строится единственный путь в остове между её концами, после чего этот путь вместе с хордой образует фундаментальный цикл. Далее реализованы:

- вычисление симметрической разности наборов рёбер;
- попытка разложения полученного чётного подграфа на простые циклы.

Метод `compute()`

Вход: исходный граф и остов, сохранённые в объекте `FundamentalCycleSystem`.

Выход: список всех фундаментальных циклов.

Описание: вычисляет систему фундаментальных циклов для выбранного остова.

Код представлен в листинге 35.

```
std::vector<FundamentalCycle> compute() const;
```

Листинг 35: Фрагмент функции `compute()`

Метод `symmetricDifference(a, b)`

Вход: два отсортированных множества рёбер, представляющих циклы.

Выход: отсортированное множество рёбер, принадлежащее ровно одному из двух входов.

Описание: находит симметрическую разность двух циклов.

Код представлен в листинге 36.

```
static std::vector<std::pair<int, int>>
symmetricDifference(
    const std::vector<std::pair<int, int>>& a,
    const std::vector<std::pair<int, int>>& b
);
```

Листинг 36: Фрагмент функции `symmetricDifference(a, b)`

Метод `decomposeIntoCycles(edges)`

Вход: множество рёбер чётного подграфа.

Выход: набор восстановленных простых циклов, если такое разложение удалось; в противном случае возвращается пустой результат.

Описание: раскладывает чётный подграф на простые циклы.

Код представлен в листинге 37.

```
static std::vector<std::vector<int>>
decomposeIntoCycles(
    const std::vector<std::pair<int, int>>& edges
);
```

Листинг 37: Фрагмент функции `decomposeIntoCycles(edges)`

Следует отметить, что в `lab5` присутствует и отдельный модуль `FundamentalCutSystem`, реализующий фундаментальные разрезы относительно остова. Однако в текущей версии меню задействована именно фундаментальная система циклов, поскольку она напрямую связана с операцией симметрической разности и наглядно демонстрирует структуру пространства циклов.

Пятая лабораторная работа соответствует пунктам меню 24–26. Пользователь может построить эйлеров обход, вывести фундаментальную систе-

му циклов и получить результат симметрической разности выбранных циклов.

7. Структура программной системы

7.1. Общая архитектура

Итоговая программа имеет модульную архитектуру. В качестве базового типа используется класс `AdjacencyGraph`, который хранит граф в виде списка смежности и предоставляет операции добавления вершин и рёбер, получения списка вершин, соседей и матрицы смежности. Для задач третьей лабораторной работы введён отдельный класс `FlowNetwork`, в котором, помимо самих дуг, хранятся пропускные способности, стоимости и текущий поток.

Основные рабочие объекты в `main.cpp` таковы:

- `currentGraph` — текущий граф, сгенерированный пользователем;
- `currentFlowNetwork` — сеть потоков, построенная по текущему ориентированному графу;
- `currentSpanningTree` — минимальный остов или остовный лес;
- `lastPruferCode` — последний полученный код Прюфера;
- `lastFundamentalCycles` — кэш фундаментальной системы циклов.

Такой подход позволяет пользователю сначала сгенерировать граф, а затем поэтапно применять к нему алгоритмы разных лабораторных работ без пересоздания данных.

7.2. Состав модулей

Таблица 1: Основные модули программы

Модуль	Назначение
Graph, Generator	Базовое представление графа, генерация рёбер и весов по биномиальному распределению, вычисление статистики степеней.
Eccentricity, ShimbellMethod, PathCounter	Метрика графа, поиск диаметральных путей, матричный метод Шимбелла, поиск всех простых путей.
BFS, Dijkstra, AlgorithmComparator	Обход графа в ширину, кратчайшие пути, сравнение алгоритмов по числу итераций.
FlowNetwork, FlowNetworkBuilder, MaxFlowSolver, MinCostFlow	Построение сети потоков, остаточная сеть, максимальный поток, поток минимальной стоимости.
Kirchhoff, Kruskal, PruferCode, MaxMatching, EdmondsMatching	Остовные деревья, кодирование деревьев, жадное и максимальное паросочетания.
EulerianCycle, FundamentalCycles, FundamentalCuts	Эйлеризация графа, эйлеров обход, базис циклов; в кодовой базе также присутствует модуль фундаментальных разрезов.
MermaidExporter	Экспорт графа, сети потоков и остова в формат Mermaid для внешней визуализации.

7.3. Пользовательский интерфейс

В файле `lab5/src/main.cpp` реализовано единое текстовое меню. Его пункты сгруппированы по лабораторным работам. Для первой лабораторной доступны генерация графа, вычисление эксцентриситетов, метод Шимбелла, поиск всех путей и вывод матриц. Для второй — BFS, алгоритм Дейкстры и сравнение алгоритмов. Для третьей — построение сети потоков, матрицы пропускных способностей и стоимостей, а также алгоритмы потоков. Для четвёртой — теорема Кирхгофа, минимальный остов, код Прюфера и паросочетания. Для пятой — эйлеров обход и работа с фундаментальной системой циклов.

Отдельно предусмотрены сервисные возможности: скрывание разделов меню и экспорт текущих структур в Mermaid. Это делает программу не только учебной, но и пригодной для наглядной демонстрации результатов.

8. Особенности и ограничения текущей реализации

Разработанный комплекс успешно охватывает все пять лабораторных работ, однако при анализе реализации важно учитывать несколько особенностей.

1. Генератор графов ориентирован на повторное использование в разных лабораторных работах и надёжно обеспечивает связность, отсутствие петель и повторяющихся рёбер. При этом строгая ацикличность гарантируется непосредственно только в ориентированном режиме за счёт добавления дуг по правилу $u < v$.
2. Алгоритм Дейкстры в теории рассчитан на неотрицательные веса. В программе пользователь технически может выбрать и отрицательные веса, поэтому при анализе второй лабораторной работы необходимо использовать корректный режим входных данных.
3. В задаче потока минимальной стоимости используется последовательный поиск кратчайших путей в остаточной сети. Реализация удобна для учебной демонстрации и трассировки шагов, но не претендует на промышленную оптимальность.
4. В кодовой базе пятой лабораторной работы присутствуют и фундаментальные разрезы, и фундаментальные циклы, однако в меню вынесена именно работа с фундаментальной системой циклов.
5. Визуализация не встроена непосредственно в консольное приложение, но предусмотрен экспорт в Mermaid, что упрощает внешнее построение схем графа, сети потоков и остова.

Заключение

В ходе выполнения лабораторных работ был создан единый программный комплекс по теории графов, объединяющий задачи генерации графов, их метрического анализа, обходов, кратчайших путей, сетей потоков, остовных деревьев, кодирования деревьев, паросочетаний и эйлеровых обходов.

Лабораторная работа №1

1. реализована генерация графа на основе биномиального распределения;
2. реализовано вычисление эксцентриситетов вершин;
3. реализован поиск центра графа;
4. реализовано определение диаметральных вершин графа;
5. реализован метод Шимбелла;
6. реализован поиск всех простых путей между двумя вершинами.

Лабораторная работа №2

7. реализован обход графа в ширину;
8. реализован классический алгоритм Дейкстры;
9. реализовано сравнение алгоритмов по числу итераций.

Лабораторная работа №3

10. реализовано построение сети потоков по ориентированному графу;
11. реализован поиск максимального потока;
12. реализован поиск потока минимальной стоимости.

Лабораторная работа №4

13. реализовано вычисление числа остовных деревьев по теореме Кирхгофа;

14. реализовано построение минимального остова алгоритмом Краскала;
15. реализовано кодирование дерева кодом Прюфера;
16. реализовано декодирование дерева по коду Прюфера;
17. реализован поиск паросочетания.

Лабораторная работа №5

18. реализована проверка графа на эйлеровость;
19. реализована эйлеризация графа при необходимости;
20. реализовано построение эйлерова обхода;
21. реализовано построение фундаментальной системы циклов;
22. реализована операция симметрической разности циклов.

Главным результатом является не просто набор отдельных решений, а интегрированная архитектура, где алгоритмы взаимно дополняют друг друга. Граф, сеть потоков, остов и цикловая структура представлены как связанные сущности, а пользователь может шаг за шагом переходить от одной лабораторной работы к другой в рамках одной программы. Такой подход делает выполненный проект удобным как для учебной демонстрации, так и для дальнейшего расширения.